Hello Java!

Let's Code Together

by Aditya Varma

Preface

Index

Introduction

//Introduction to Java

What is Java?

Java is a high-level, general-purpose programming language used to build everything from small applications to massive enterprise systems. You can think of Java as a universal translator, you write code once, and it automatically adapts to different devices and operating systems.

Java belongs to the category of compiled languages.
This means:

- You write code in Java.
- A compiler converts it into a special format called bytecode.
- This bytecode is then executed by the Java Virtual Machine (JVM) on any device.

You can imagine Java's compiler as a chef who prepares a basic dish (bytecode), and the JVM is like local cooks in each country adding their own flair depending on their kitchen (OS/platform).
The recipe stays the same, only the cooking varies.

Why is bytecode important?

Because bytecode is not tied to any specific device. It's a "universal" format that works everywhere, that's where Java gets its magic.

Features of Java!

1. Platform Independence

Java's famous idea is "Write Once, Run Anywhere" (WORA).

This means:

- You write Java code once.
- It gets converted into bytecode.
- That bytecode runs on any system where a JVM exists — Windows, Linux, macOS, Android, etc.

Think of bytecode like a movie subtitle file, the same subtitle file works on many different media players. Each player interprets it in its own way, but you don't need to create separate subtitles for each one. This reduces work for programmers, no need to maintain different code versions for each platform.

2. OOP Language (Object-Oriented Programming)

Java uses the object-oriented approach, which means it organizes programs into "objects", mini units that combine data and functions.

You can imagine objects like real-world things:

- A Car object has data (color, model) and actions (start, brake).
- A Student object has data (name, age) and actions (study, takeExam).

OOP helps keep programs clean, organized, and easier to expand.

3. Secure & Robust

Java takes security seriously.

It avoids many common programming hazards because:

- Memory management is automatic
- Many risky operations are restricted
- Programs run inside the JVM "sandbox," giving an extra layer of protection

Java also does automatic memory handling:

- When you create an object, Java automatically allocates memory for it.
- When the object is no longer used, the Garbage Collector recycles that memory.

You can think of it like a house where a cleaning robot (GC) quietly removes stuff you no longer need, preventing a messy room (memory leak).

This makes Java stable (robust) and safe.

4. Portable

Java programs can move from one computer to another without modification.

Since the bytecode stays the same everywhere, Java apps don't care whether they're running on:

- A laptop
- A server
- A mobile device
- A smart TV

It's as portable as carrying a USB drive with universal files plug in anywhere, and it works.

What Java is Used For?

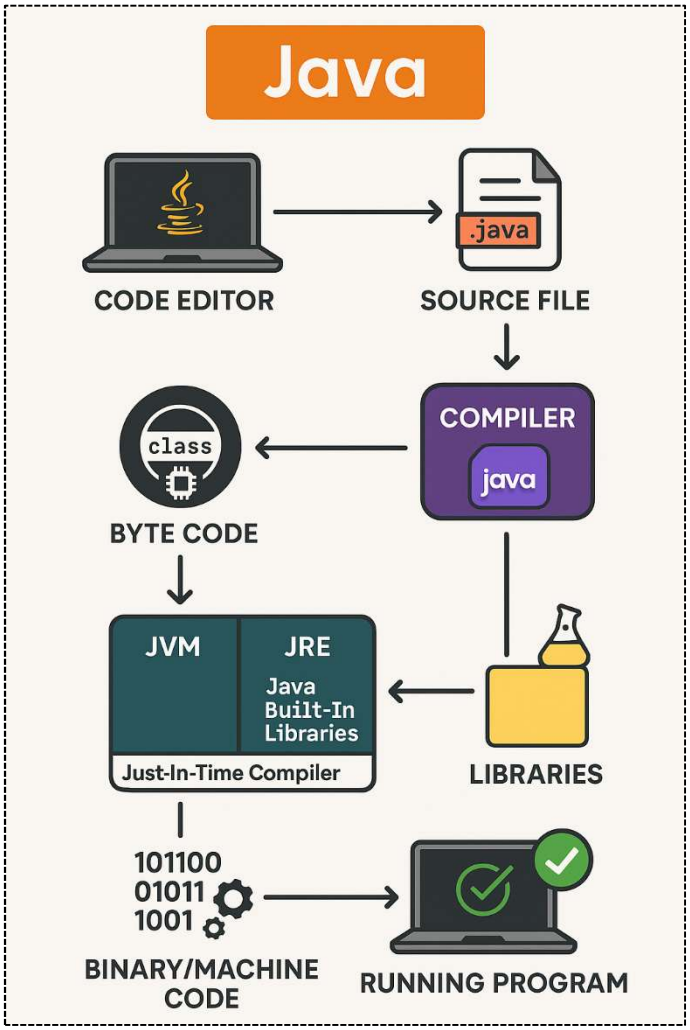
Java has been around for decades and still powers a huge part of the software world. It's used in:

- 💼 Enterprise software (banking systems, ERPs, corporate tools)
- 📱 Android apps (Java was the primary Android language for years)
- 🌐 Web applications (back-end servers using Java)
- ☁️ Cloud-based systems
- 🎮 Games (lightweight games and certain engines)
- 🖥️ Desktop applications

- 🔌 Embedded systems (smart cards, sensors, appliances)

Basically, Java is like a Swiss Army Knife — you'll find it almost everywhere in the tech world.

Write Once, Run Anywhere



//JVM, JRE & JDK – The Three Pillars of Java

When learning Java, these three names show up everywhere. They sound technical, but once you see them as parts of a mini Java ecosystem, everything becomes easy to understand.

Let's break them down with a simple analogy:

1. JVM – Java Virtual Machine

Think of the JVM as a translator who understands bytecode.

- You give JVM the bytecode produced by the compiler.
- JVM reads it, understands it, and turns it into instructions your computer can execute.
- Each operating system has its own version of the JVM, but they all understand the same bytecode.

Simple Analogy:

Imagine you write a letter in a universal language. The JVM is like a local interpreter who reads the letter and speaks in the local language (Windows, Linux, Mac). That's how Java achieves Write Once, Run Anywhere.

2. JRE – Java Runtime Environment

The JRE is the "playground" where Java programs actually run.

It includes:

- The JVM
- Built-in Java libraries (pre-written code you can use)
- Tools needed to run programs

But note: You cannot write or compile Java code with the JRE. It's only for running applications.

Analogy:

If JVM is the performer, then JRE is the stage + lights + tools that allow the performer to do their job.

3. JDK – Java Development Kit

The JDK is the complete toolset for people who want to write Java programs.

It contains:

- Everything inside the JRE
- Plus, tools for developers
    - Java compiler (javac)
    - Debuggers
    - Documentation tools

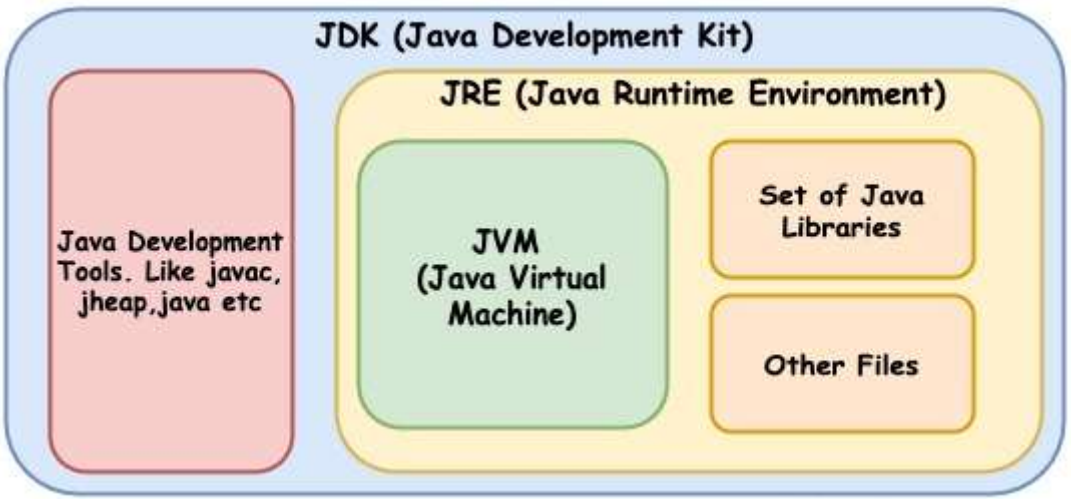
Analogy:

If JRE is the stage, then JDK is the entire theatre building, including:

- Stage (JRE)
- Backstage tools

- Equipment
- Workshops

JDK = Everything you need to create, compile, and run Java code.



//Installing Java (JDK)

Check if Java is already installed:

On Windows:

1. Open Command Prompt (CMD)
2. Type:

```
java -version
```

If Java is installed, you will see something like:

```
java version "17.0.10"
```

If Java is NOT installed, you will see:

```
'java' is not recognized as an internal or external command
```

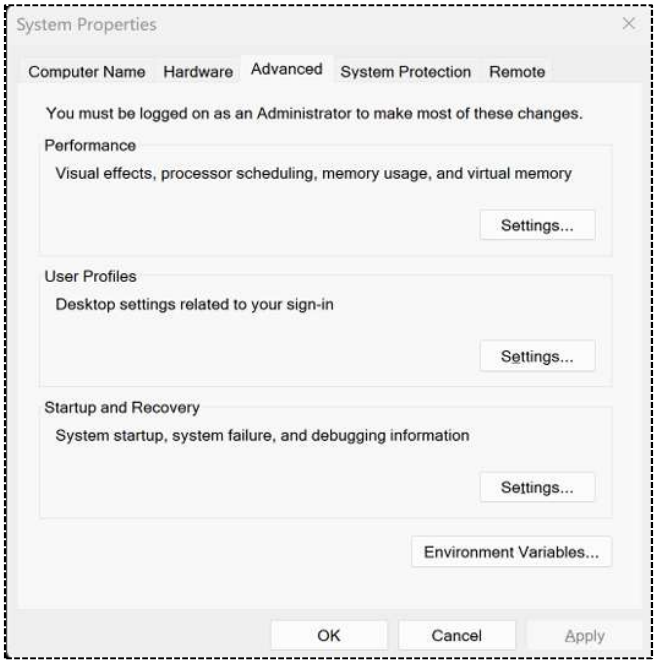
Installing Java (JDK):

1. Visit the Oracle Java SE Downloads page at https://www.oracle.com/java/technologies/downloads/.
2. Select the appropriate installer for your Windows operating system (for example, Windows x64 Installer).
3. Download the installer by clicking on the file link.
4. Run the downloaded installer (the .exe file) and follow the on-screen instructions to complete the installation.

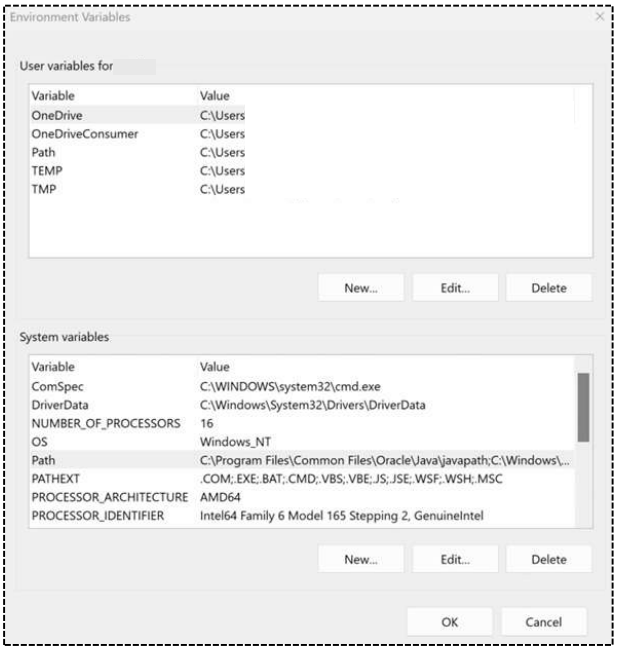
Setting Up Environment Variables:

(This step is sometimes done automatically by newer installers, but you should know it.)

1. Find the JDK installation path:
   Example path:
   `C:\Program Files\Java\jdk-25`
2. Add JAVA_HOME:
   - Search “Edit the system environment variables” -> Click Environment Variables
   - Under System variables, click New
   - Variable name: JAVA_HOME
   - Variable value: (your JDK folder path): C:\Program Files\Java\jdk-25



3. Add JDK to PATH:
    - Go to System variables → Path → Edit
    - Click New, add: `%JAVA_HOME%\bin`



4. Verify installation:
    - Open CMD and type:

```
java -version
```

You should see version numbers.

//Installing an IDE (VS Code / IntelliJ)

Option A: VS Code

1. Download and install VS Code
2. Open VS Code → Go to Extensions
3. Install:
    - Extension Pack for Java (important)
    - Debugger for Java
    - Java Test Runner (optional)

That's it VS Code is ready for Java development.

Option B: IntelliJ IDEA

(More beginner-friendly than VS Code. Handles everything automatically.)

1. Download IntelliJ IDEA Community Edition (free)
2. Install and open it
3. IntelliJ automatically detects your JDK
    - If it asks: just select the JDK folder (example: C:\Program Files\Java\jdk-17)
4. Create a New Java Project and start coding.

//Creating & Running Your First Java Program

Using CMD (Command Prompt):

Step 1: Create a new file named Hello.java in notepad.

Java requires the filename and class name to be the same.

Step 2: Write this code in the notepad,

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Step 3: Save the file in a folder you want.

Step 4: Open CMD and go to that folder.

```
cd C:\JavaPrograms
```

Step 5: Compile the file

```
javac Hello.java
```

This will create:

Hello.class (Your bytecode file)

Step 6: Run the program

```
java Hello
```

Output:

```
Hello World
```

Using IntelliJ IDEA (recommended):

Step 1: Open IntelliJ → Create New Project

- Select New Project
- Choose Java
- Select JDK 17
- Click Create

Step 2: Create a Java File

Inside src folder:

1. Right-click src
2. Select New → Java Class
3. Name it:

Step 3: Write the Program

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Step 4: Run the Program

At the top-left of the editor you will see a green play button ▶. You will get the Output.

//Basic Structure of a Java Program

Every Java program follows a specific structure. Once you understand this structure, Java becomes much easier to read and write.
Let's break it down part by part.

Understanding Class Structure:

Every Java file contains **at least one class**.

General Structure:

```
public class ClassName {
    // variables (optional)
    // methods (optional)
}
```

Points to remember:

- The class name must match the file name
  e.g., file: Hello.java → class: Hello
- A class is like a container that holds your code.
- Everything in Java lives inside a class.

Example: Think of a class as a **house**, and all your code sits inside it.

The main() Method Explained:

This is the starting point of **every** Java program.

```
public static void main(String[] args) {
    // program starts here
}
```

What each word means:

- public → Anyone can run this method
- static → Runs without creating an object
- void → Does not return anything
- main → Special method where program starts
- String[] args → Accepts inputs from the user (command-line arguments)

Think of main() as:

The **front door** of your program. Java starts running your code by entering through this door.

Compiling & Running a Java Program:

Compilation: Java code (Hello.java) → compiled into bytecode → Hello.class

javac Hello.java

Execution: Run the bytecode using the JVM

java Hello

Common Errors Beginners Face:

1. File name does not match class name:

   public class Hello {}

   but file is named: HelloWorld.java

   Fix: Rename file to Hello.java

2. Missing semicolon
3. Wrong capitalization:

   Java is **case-sensitive**.

   System ≠ system
   main ≠ Main
   String ≠ string

   Fix: Use exact uppercase/lowercase.
4. Missing curly braces { }:

   Every opening brace { must have a closing brace }.
5. Running the program with .java extension:

   You never include the .java when running.
6. Typing code outside the class:

   Everything must be **inside the class** and mostly inside the **main method** for beginners.

Quick Visual Summary:

```
public class Hello {        ← Class begins
    public static void main(String[] args) {   ← Main method begins
        System.out.println("Hello World");     ← Statement
    }                                          ← Main closes
}                                              ← Class closes
```

//Understanding Bytecode & Java Compilation Process

When you write a Java program, you type code in English-like syntax inside a .java file. But your computer cannot understand Java directly. Computers understand machine code (0s and 1s).
So, Java uses a two-step process that makes it special:

1. Java Compilation: From .java → .class
Whenever you compile a Java file using:

```
javac Program.java
```

the Java compiler (JAVAC) converts your source code into bytecode, stored in a .class file.

What is Bytecode?

- Bytecode is a universal, platform-independent code.
- It is not machine code, and not Java code — it is something in between.
- Every Java program is first converted to bytecode.

Why does Java use bytecode?

Because bytecode can run on any operating system — Windows, macOS, Linux — as long as the system has a JVM. This is why Java is called:

"Write Once, Run Anywhere (WORA)"

2. JVM Execution: From Bytecode → Machine Code
Once bytecode is created, the Java Virtual Machine (JVM) takes over.
When you run:

```
java Program
```

JVM loads the .class file and performs two main tasks:

1. Bytecode Verification
Checks if the bytecode is:

- safe
- valid
- not trying to access memory illegally
- secure for execution

2. Bytecode Execution
JVM converts bytecode into machine code specific to your operating system and CPU.
Different systems have different JVMs:

- Windows JVM
- Linux JVM
- macOS JVM

But all understand the same bytecode, which is the magic of platform independence.

How the Full Process Looks:

Here is the entire journey of a Java program simplified:

Your Code (.java)
|
COMPILED BY
|
javac (Java Compiler)
↓
Bytecode (.class)
|
EXECUTED BY
|
JVM (Java Virtual Machine)
↓
Machine Code (CPU-readable)

Why This Process Is Amazing:

✓ Portable — runs on any device
✓ Secure — JVM checks code before running
✓ Optimized — JVM improves performance at runtime
✓ Reliable — same program behaves the same everywhere

Java Basics

// Java Tokens

When you read a sentence, your brain breaks it into small parts—words, symbols, punctuation. Java also does the same. A Java program is built using tiny building blocks called *tokens*. These are the smallest units that still have meaning in code.

Think of tokens like LEGO pieces—small on their own, but powerful when connected. Java has 4 major types of tokens:

1. Keywords
2. Identifiers
3. Literals
4. Symbols & Operators

Let's explore them one by one.

Keywords:

Keywords are special words that Java has reserved for its own use. You cannot use them as variable names or class names because Java already knows what these words mean.

Examples:

class, public, static, int, void, if, else, return

Analogy:

Think of keywords as "VIP seats" in a theatre—reserved and untouchable.

Identifiers:

Identifiers are the names you give to things in your program:

- Class names
- Variable names
- Method names
- Object names

Rules (easy to remember):

- Can use letters, digits, _, $
- Cannot start with a digit
- Cannot use spaces
- Cannot use keywords
- Case-sensitive (Name, name, and NAME are different)

Identifiers are like labels you stick on your storage boxes so you know what's inside.

Literals:

Literals are fixed values written directly in the code. They don't change.

Types of literals:

- 10 → integer literal
- 3.14 → floating literal
- 'A' → character literal
- "Hello" → string literal
- true, false → boolean literal

These are the raw values that your program works with.

Symbols & Operators:

These are characters that help Java understand code structure and perform actions.

Common symbols:

- ( ) → parentheses
- {} → braces
- [ ] → brackets
- ; → statement end
- " " → string quotes

Operators do the work:

- Arithmetic: +, -, *, /, %
- Relational: ==, !=, >, <
- Logical: &&, ||, !
- Assignment: =, +=, -=

Think of operators as the tools your program uses to get things done.

Why Tokens Matter?

Tokens are the foundation of Java syntax. If Java were a language, tokens would be its alphabet. Understanding them helps you read, write, and debug programs correctly.

Lesson Program: Tokens in Java.

Source Code:

Tokens.java

```java
// Program: Tokens in Java

public class Tokens {
    public static void main(String[] args) {

        System.out.println("\n=== TOKENS IN JAVA ===");

        System.out.println("""
Java programs are made of tokens:
1. Keywords
2. Identifiers
3. Literals
4. Symbols & Operators
""");

        System.out.println("Keyword Example: public, class, static");
        System.out.println("Identifier Example: TokensInJava, count, value");

        System.out.println("Literals:");
        System.out.println(10);
        System.out.println(3.14);
        System.out.println('A');
        System.out.println("Hello Java");
        System.out.println(true);

        System.out.println("Symbols & Operators:");
        System.out.println("Semicolon ; ends statements");
        System.out.println("Using + operator: " + ("Java" + " Tokens"));
    }
}
```

//Comments in Java

When programmers write code, they also leave notes to themselves or others—just like writing sticky notes on a book page.
In Java, these notes are called comments, and the important part is:

The computer completely ignores them. They exist only for humans.

Why do we use comments?

- To explain the logic
- To make code readable
- To give instructions or warnings
- To turn OFF a line temporarily without deleting it

Java gives us 3 types of comments:

Single-Line Comment (//)

A single-line comment starts with //. Everything after it on the same line gets ignored.
Used for:

- Quick notes
- Explaining one line
- Temporarily disabling a line

Think of it as whispering a small reminder in your code.

Multi-Line Comment (/* … */)

Starts with /* and ends with */. Anything between them is ignored.
Used for:

- Long explanations
- Notes across multiple lines
- Disabling a block of code

Think of it as wrapping text inside a "big comment box."

Documentation Comment (/ … */)**

This is a special style used for generating official Java documentation using the javadoc tool.
Inside it, special tags are used:

- @author
- @version
- @param
- @return

These are used mostly in professional Java projects. Think of them as comment blocks that turn into real documentation pages.

Lesson Program: Comments in Java.

Source Code:

Comments.java

```java
// Program: Comments in Java

public class Comments {
    public static void main(String[] args) {

        System.out.println("\n=== COMMENTS IN JAVA ===");

        System.out.println("""
Types of comments:
1. Single-line
2. Multi-line
3. Documentation comments
""");

        // Single-line comment example
        System.out.println("Single-line comment example printed.");

        /*
           This is a multi-line comment.
           Used for long explanations.
        */
        System.out.println("Multi-line comment example printed.");

        /**
         * Documentation comments help create official docs.
         */
        System.out.println("Documentation comment example printed.");
    }
}
```

//Naming Conventions in Java

Naming conventions are simply rules for naming things in your Java program. They don't affect how the code runs, but they affect how it looks, how easy it is to understand, and how professional it feels. Java programs contain names for:

- classes
- methods
- variables
- constants
- packages
- interfaces

Because Java is case-sensitive, following naming conventions becomes very important. These conventions ensure your code is: Clean, Predictable, Easy to understand, Standardized across developers

Let's explore each naming rule one by one.

1. Class Naming Convention

Classes represent objects or entities, so their names should be nouns. Use PascalCase (UpperCamelCase), First letter of every word is capital, Name must be meaningful

Examples (correct):

- StudentRecord
- OrderDetails
- PaymentGateway

2. Method Naming Convention

Methods perform actions, so they should be verbs. Use camelCase, first word lowercase, next words start with uppercase

Correct:

- calculateTotal()
- getName()
- displayMessage()

3. Variable Naming Convention

Variables hold values. Their names must be clear and descriptive. Use camelCase, start with a letter, not a digit, avoid very short names (except i, j in loops), Should be easy to understand

Correct:

- studentAge
- totalMarks
- numberOfItems

4. Constant Naming Convention

Constants represent fixed values. Use UPPERCASE, use underscores between words, declare them using final

Correct:

- MAX_SPEED
- PI_VALUE

5. Package Naming Convention

Packages represent folder structures. Use all lowercase, Use meaningful names, Companies use reverse domain naming

Correct:

- student.management
- com.school.app

6. File Naming Convention

A Java file must match the public class name.

Lesson Program: Naming Conventions in Java.

Source Code:

Comments.java

```java
// Program: Java Naming Conventions

public class NamingConvention {
    public static void main(String[] args) {

        System.out.println("\n=== JAVA NAMING CONVENTIONS ===");

        System.out.println("""
1. Class → PascalCase
2. Method → camelCase
3. Variable → camelCase
4. Constant → UPPER_CASE
5. Package → lowercase
6. Interface → PascalCase
""");

        System.out.println("Correct Class Name: StudentData");
        System.out.println("Correct Method Name: calculateMarks()");
        System.out.println("Correct Variable Name: totalPrice");
        System.out.println("Correct Constant Name: MAX_LIMIT");
        System.out.println("Correct Package Name: student.management");
    }
}
```

//Escape Sequence in Java

When we write text inside System.out.println(), we usually print normal characters like letters, digits, and symbols.
But sometimes we need to print:

- a new line
- a tab space
- a double quote inside the string
- a backslash
- specially formatted text

Normal characters cannot do this directly. So Java gives us special character combinations called escape sequences.

What are Escape Sequences?

Escape sequences are special characters written using a backslash (\). They allow Java to print characters that are otherwise difficult or impossible to write directly.
Example:

\" prints a double quote
\\ prints a backslash
\n prints a new line

Common Escape Sequences in Java

1. \n New Line
   - Moves the output to the next line.
   - Works like pressing the Enter key.
2. \t Tab Space
   - Inserts a horizontal tab (same as pressing TAB).
   - Used to align data neatly.
3. \" Double Quote
   - Allows printing " inside a string.
   - Without escape sequence, Java thinks the string ends.
4. \\ Backslash
   - Prints a single backslash.
   - Because a single backslash starts an escape sequence, we need double backslash.

Why Escape Sequences Are Useful?

- They help in:
- Pretty formatting
- Writing multi-line messages
- Printing quotes inside text
- Drawing patterns
- Making console output cleaner

Escape sequences make your output professional and readable.

Lesson Program: Escape Sequence in Java.

Source Code:

EscapeSequences.java

```java
// Program: Escape Sequences in Java

public class EscapeSequences {
    public static void main(String[] args) {

        System.out.println("Line1\nLine2");          // \n -> New line
        System.out.println("Name:\tJohn");          // \t -> Tab
        System.out.println("He said, \"Hello\"");    // \" -> Double quote
        System.out.println("Path: C:\\Files");       // \\ -> Backslash

        // Combined example
        System.out.println("Start\n\tMiddle\n\t\tEnd");
    }
}
```

// Variables and Constants

In any Java program, we need a way to store information — like a number, a name, or a calculated result. Java provides two main ways to store data:

1. Variables
2. Constants (using final)

Let us understand both:

1. What is a Variable?

A variable is like a *container* that stores a value. The value can change while the program is running.

Example:

- A variable can be like a "score" in a game, it keeps changing.
- Or "age", it updates every year.

Features of Variables:

- Can store different types of data
- Value can be changed anytime
- Must have a data type
- Requires a name (follows naming conventions)

2. Declaring a Variable:

Declaration = telling Java what type of data the variable will hold. At this stage, Java only reserves memory, no value yet.

Syntax:

```
dataType variableName;
```

Examples:

```
int age;
double price;
String city;
```

3. Initializing a Variable

Initialization = giving the **first value** to the variable.

Syntax:

```
variableName = value;
```

Examples:

```
age = 25;
price = 199.99;
city = "Mumbai";
```

You can also declare and initialize in one line:

```
int number = 100;
```

4. Using Variables

Once declared and initialized, we can use variables in:

- printing
- calculations
- expressions
- method calls

Variables make programs flexible and dynamic.

5. Constants in Java (final)

A constant is a value that never changes once assigned. We create a constant using the final keyword.

Features of Constants:

- Value cannot be changed
- Must be assigned only once
- Written in UPPERCASE (naming convention)
- Used for fixed values like PI, speed limit, max range, etc.

Syntax:

```
final dataType CONSTANT_NAME = value;
```

Examples:

```
final int MAX_USERS = 500;
final double PI = 3.14159;
```

If you try to change a constant later, Java gives an error.

6. Difference Between Variables and Constants

Feature	Variable	Constant
Value	Can change	Cannot change
Keyword	None	final
Naming	camelCase	UPPER_CASE
Usage	flexible values	fixed permanent values

7. Why variables & constants matter?

They help programs become:

- dynamic
- flexible
- readable
- maintainable

Lesson Program: Variables and Constants in Java.

Source Code:

VariablesAndConstants.java

```java
// Program: Variables and Constants in Java

public class VariablesAndConstants {
    public static void main(String[] args) {

        // variable declaration + initialization
        int age = 25;
        double price = 199.99;
        String name = "John";

        // using variables
        System.out.println("Age: " + age);
        System.out.println("Price: " + price);
        System.out.println("Name: " + name);

        // constants using final
        final int MAX_STUDENTS = 100;
        final double PI = 3.14159;

        System.out.println("Max Students: " + MAX_STUDENTS);
        System.out.println("PI Value: " + PI);

        // variable changes allowed
        age = 26;
        System.out.println("Updated Age: " + age);

        // MAX_STUDENTS = 200; // ❌ Not allowed (constant)
    }
}
```

// Data Types in Java

In Java, every variable must have a data type. A data type tells Java what kind of value a variable will store, a number, decimal, character, or text.
Java data types are divided into two main groups:

1. Primitive Data Types
2. Non-Primitive (Reference) Data Types

Let's understand both:

1. Primitive Data Types

Primitive types are the simplest and most basic data types in Java. They store values directly in memory and are very fast.
Java has 8 primitive data types:

Type	Size	Description
byte	1 byte	Very small integers (-128 to 127)
short	2 bytes	Small integers
int	4 bytes	Most used integer type
long	8 bytes	Large integers (needs L)
float	4 bytes	Decimal values (needs f)
double	8 bytes	Default decimal type
char	2 bytes	Single character ('A', '#')
boolean	1 bit	true/false

Why use primitive types?

- They are fast
- They use fixed memory
- Good for calculations, numeric logic, flags, etc.

2. Non-Primitive (Reference) Data Types

These are more complex types. They do not store values directly. Instead, they store the memory address (reference) of an object.
Common non-primitive types include:

- String
- Arrays
- Classes
- Objects
- Interfaces

Characteristics:

- Begin with uppercase (String, Integer)
- Can hold multiple values
- Can be null
- Have built-in methods
- Size is not fixed

3. Difference: Primitive vs Non-Primitive

Feature	Primitive	Non-Primitive
Stores	Actual value	Memory address (reference)
Size	Fixed	Not fixed
Starts with	lowercase	Uppercase
Methods	No	Yes
Speed	Faster	Slower

Understanding this difference is important because memory and performance depend on it.

Lesson Program: Data Types in Java.

Source Code:

DataTypes.java

```java
// Program: Data Types in Java

public class DataTypes {
    public static void main(String[] args) {
        // primitive data types
        byte b = 10;
        short s = 200;
        int num = 5000;
        long bigNum = 123456789L;
        float f = 5.75f;
        double d = 99.99;
        char ch = 'J';
        boolean flag = true;

        System.out.println("byte: " + b);
        System.out.println("short: " + s);
        System.out.println("int: " + num);
        System.out.println("long: " + bigNum);
        System.out.println("float: " + f);
        System.out.println("double: " + d);
        System.out.println("char: " + ch);
        System.out.println("boolean: " + flag);

        // non-primitive types
        String name = "John Doe";
        int[] marks = {90, 85, 88};

        System.out.println("String: " + name);
        System.out.println("String length: " + name.length());
        System.out.println("Array marks: " + marks[0] + ", " + marks[1] + ", " + marks[2]);
    }
}
```

// Primitive Data Types in Java

Primitive data types are the most basic building blocks in Java. They are called "primitive" because they store simple, direct values and not objects.

Why are primitive types important?

Because they help us store:

- Whole numbers (like age, marks)
- Decimal values (like price, salary)
- Characters (like 'A', '5')
- True/false conditions

Key Features of Primitive Types

- Stores actual value directly (not memory address)
- Very fast and memory-efficient
- Names always start with lowercase (int, char, double)
- Size is fixed (decided by JVM)
- Cannot call methods on them

The 8 Primitive Data Types:

1. byte (1 byte)
   - Smallest integer type
   - Range: –128 to 127
   - Used when we want to save memory
   - Example: byte age = 25;

2. short (2 bytes)
   - Slightly bigger than byte
   - Range: –32,768 to 32,767
   - Example: short temperature = 300;

3. int (4 bytes)
   - Most commonly used
   - Range: large enough for most tasks
   - Default integer type
   - Example: int marks = 92;

4. long (8 bytes)
   - For huge numbers
   - Must end with L
   - Example: long population = 8000000000L;

5. float (4 bytes)
   - Decimal numbers with ~7 digits precision
   - Must end with f
   - Example: float price = 55.75f;

6. double (8 bytes)
   - Most commonly used decimal type
   - High precision (~15 digits)
   - Example: double salary = 45000.99;

7. char (2 bytes)
   - Stores one character
   - Uses single quotes
   - Supports Unicode values too
   - Example:

```
char letter = 'J';
char unicodeA = 65; // Prints 'A'
```

8. boolean (1 bit)
   - Stores true or false
   - Used in conditions
   - Example: boolean isJavaFun = true;

Default values (only when used as class fields)

Type	Default
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0
char	'\u0000'
boolean	false

Lesson Program: Primitive Data Types in Java.

Source Code:

PrimitvieDataTypes.java

```java
// Program: Primitive Data Types

import java.util.Scanner;

public class PrimitiveDataTypes {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // byte
        byte b = 100;
        System.out.println("byte: " + b);
        System.out.print("Enter byte: ");
        byte userByte = sc.nextByte();
        System.out.println("You entered: " + userByte);

        // short
        short s = 30000;
        System.out.println("short: " + s);
        System.out.print("Enter short: ");
        short userShort = sc.nextShort();
        System.out.println("You entered: " + userShort);

        // int
        int i = 95;
        System.out.println("int: " + i);
        System.out.print("Enter int: ");
        int userInt = sc.nextInt();
        System.out.println("You entered: " + userInt);

        // long
        long l = 8000000000L;
        System.out.println("long: " + l);
        System.out.print("Enter long: ");
        long userLong = sc.nextLong();
        System.out.println("You entered: " + userLong);

```

```
38          // float
39          float f = 99.75f;
40          System.out.println("float: " + f);
41          System.out.print("Enter float: ");
42          float userFloat = sc.nextFloat();
43          System.out.println("You entered: " + userFloat);
44
45          // double
46          double d = 55000.456;
47          System.out.println("double: " + d);
48          System.out.print("Enter double: ");
49          double userDouble = sc.nextDouble();
50          System.out.println("You entered: " + userDouble);
51
52          // char
53          char c1 = 'J';
54          char c2 = 65;
55          System.out.println("char 1: " + c1);
56          System.out.println("char 2: " + c2);
57          System.out.print("Enter char: ");
58          char userChar = sc.next().charAt(0);
59          System.out.println("You entered: " + userChar);
60
61          // boolean
62          boolean flag = true;
63          System.out.println("boolean: " + flag);
64          System.out.print("Enter boolean: ");
65          boolean userBoolean = sc.nextBoolean();
66          System.out.println("You entered: " + userBoolean);
67
68          sc.close();
69      }
70  }
```

// Input and Output in Java

Input and Output (I/O) are fundamental in programming because programs interact with users or other systems. **Input** refers to taking data from the user, files, or other sources. **Output** refers to displaying data to the user, storing results in files, or sending data to other systems.

1. Output in Java

Java provides built-in mechanisms to display information using the System.out object.
System.out.println() : Prints the text and moves to the next line.
System.out.print() : Prints text but stays on the same line.
System.out.printf() : Used for formatted output.

- Format specifiers:
  - %d → integer
  - %f → decimal
  - %s → string
  - %c → character
- Example:

```
System.out.printf("Marks: %d", 90);
```

2. Input in Java (Scanner Class)

Java takes input from keyboard, files, or command-line arguments. Most common method: Scanner class from java.util package.
You must import it:

```
import java.util.Scanner;
```

Then create a Scanner object:

```
Scanner sc = new Scanner(System.in);
```

Common Scanner methods

- nextInt() → reads integer
- nextDouble() → reads decimal
- next() → reads one word
- nextLine() → reads full sentence
- nextBoolean() → true/false

nextLine() Issue

If you use nextInt() first, it leaves a newline in the buffer. So nextLine() may capture an empty string.
Solution:

```
sc.nextLine();  // clear buffer
```

3. Command-Line Arguments

These are values you pass while running the program from terminal:

```
java MyProgram Hello 10
```

They are stored in: `String args[]`

Difference:

- `Scanner` → input during program
- `args[]` → input before program starts

4. Common Input Errors

1. Entering text when the program expects a number.
2. Forgetting `import java.util.Scanner;`
3. `nextLine()` not working due to leftover newline.
4. Using wrong input method for wrong data type.
5. Reading a character incorrectly (correct: `next().charAt(0)`).

Lesson Program: Input Output in Java.

Source Code:

InputOutput.java

```java
// Program: Input and Output in Java

import java.util.Scanner;

public class InputOutput {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // ---------------- Output Examples ----------------
        System.out.println("Welcome to Java I/O Demo!");
        System.out.print("This is printed on the same line. "); // System.out.print stays on the same line
        System.out.println("Now we move to the next line.");   // System.out.println moves to next line
        System.out.printf("Formatted number: %.2f\n", 123.456); // printf example for formatted output

        // ---------------- Input Examples ----------------
        // Example 1: Reading a String
        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        // Example 2: Reading an integer
        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        // Example 3: Reading a double
        System.out.print("Enter your height in meters: ");
        double height = sc.nextDouble();

        sc.nextLine(); // clear buffer before reading nextLine()

        // Example 4: Reading a full sentence
        System.out.print("Enter your city: ");
        String city = sc.nextLine();

        // Example 5: Reading a boolean
        System.out.print("Are you a student? (true/false): ");
        boolean isStudent = sc.nextBoolean();

        sc.nextLine(); // clear buffer before nextLine()

        // Example 6: Reading multiple words (full line)
        System.out.print("Enter your favorite quote: ");
        String quote = sc.nextLine();

        // ---------------- Display Collected Inputs ----------------
        System.out.println("\n===== USER DETAILS =====");
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.printf("Height: %.2f meters\n", height);
        System.out.println("City: " + city);
        System.out.println("Student: " + isStudent);
        System.out.println("Favorite Quote: " + quote);

        sc.close();
    }
}
```

// Java Basics Summary

1. Tokens
   - Smallest units of Java: Keywords, Identifiers, Literals, Operators, Separators.
   - Form the building blocks of a program.
2. Keywords & Identifiers
   - Keywords: Reserved words (int, class, if)
   - Identifiers: Names for classes, variables, methods. Must follow rules, case-sensitive.
3. Comments
   - Single-line: //
   - Multi-line: /* ... */
   - Documentation: /** ... */
   - Improves readability; ignored by compiler.
4. Escape Sequences
   - Special sequences starting with \:
     - \n → new line, \t → tab, \" → double quote, \\ → backslash.
5. Naming Conventions
   - Class: PascalCase → StudentData
   - Method: camelCase → calculateSalary()
   - Variable: camelCase → totalMarks
   - Constant: UPPERCASE → MAX_SPEED
   - Package: lowercase → student.management
   - Interface: PascalCase → Runnable
   - File name = public class name.
6. Variables & Constants
   - Variable: Stores changeable data → int age = 25;
   - Constant: Fixed value using final → final int MAX = 100;
7. Data Types
   - Primitive: byte, short, int, long, float, double, char, boolean
   - Non-Primitive: String, arrays, objects, classes
   - Primitive stores values; non-primitive stores references.
8. Input & Output
   - Output: System.out.print(), System.out.println(), System.out.printf()
   - Input: Scanner → nextInt(), nextDouble(), nextLine(), nextBoolean()
   - Handle buffer correctly when mixing numeric and string input.
   - Command-line args: Input before execution via String[] args.

Operators in Java

In programming, we constantly perform actions like calculation, comparison, decision-making, and updating values. To do these operations, Java uses operators.

What are Operators?

Operators are special symbols that tell Java to perform an operation on values (operands). They work on variables, constants, and expressions.

Example in real life:

- + means add two items
- > means compare two values

Just like in mathematics, Java uses symbols to perform actions in a program.

Why Operators Are Important?

Operators allow Java to:

- Perform mathematical calculations
- Make comparisons and decisions
- Join conditions (true/false)
- Modify values of variables
- Assign values efficiently
- Work at low-level (bits) when needed

Without operators, a program cannot calculate, decide, or interact logically.

Classification of Operators in Java

Type	Purpose
Arithmetic	Mathematical operations
Assignment	Store and update values
Relational (Comparison)	Compare values (true/false)
Logical	Combine conditions
Unary	Operates on one value only
Ternary	Short form of if-else
Bitwise	Works on individual bits (low-level programming)

Lesson Program: Operators in Java.

Source Code:

OperatorsInJava.java

```java
// Program: Operators in Java

public class OperatorsInJava {
    public static void main(String[] args) {

        int a = 10, b = 3;

        // arithmetic
        System.out.println("a + b = " + (a + b));
        System.out.println("a - b = " + (a - b));

        // assignment
        int x = 5;
        x += 3;
        x *= 2;
        System.out.println("x value: " + x);

        // relational
        System.out.println("a > b: " + (a > b));
        System.out.println("a == b: " + (a == b));

        // logical
        System.out.println("(a > b) && (a == 10): " + ((a > b) && (a == 10)));
        System.out.println("(a < b) || (b == 3): " + ((a < b) || (b == 3)));

        // unary
        int n = 5;
        System.out.println("++n: " + (++n));
        System.out.println("n--: " + (n--));
        System.out.println("n now: " + n);

        // ternary
        int age = 16;
        String result = (age >= 18) ? "Adult" : "Minor";
        System.out.println(result);

        // bitwise
        int p = 5, q = 3;
        System.out.println("p & q = " + (p & q));
        System.out.println("p | q = " + (p | q));
    }
}
```